Transformer 学习笔记 | Attention Is All You Need

> 目录：
>
> 1. transformer的基本功能、输入、输出、参量、超参量，以及损失函数的类型、输入、输出，怎么优化
> 2. 各个注意力结构总体的共性数学原理（QVK是什么，W_Q,W_V,W_K的使用以及相应QVK的生成、QVK分别代表着什么以及是如何由QVK慢慢转化为关注力权重表A再乘以V变成Z的等等）
> 3. 什么是多头，多头的知识点
> 4. 模型中不同类型的注意力模型（自注意力，掩码自注意力，交叉注意力）的数学原理与细节
> 5. 编码器和解码器的内部架构
> 6. 子层中的残差连接、前馈网络与归一化知识
> 7. 模型中蕴含的技术细节（结尾符的训练、生成的英文token数m训练和实际预测时的不同等等）

1. 模型在做什么

1.1 基本功能、输入和输出

生成文本，可以进行文本到文本之间的转换，也可以进行基于某文本对新文本未知内容的预测，论文中的主要功能是实现语言间的翻译：

Transformer 把原文（如中文）处理成数字表示，再根据原文和已经写出的译文（如英文），逐个预测下一个目标 token。

（并非是词语对词语的傻瓜式翻译，而是同时结合了英语的表达习惯和中文含义的翻译，如"我昨天吃了晚饭"--"I ate dinner yesterday"，词语语序会结合英文表达习惯变动）

1.2 总体模型的（主要）参数和超参数

参数：各层的W_Q,W_B,W_K;多头合并通过的全连接层的权重；token最开始由文字转成数字向量的嵌入表；由数字向量转成词汇表中各词分数的权重表

超参数：层数，各层生成QKV的规则，注意力的头数等等

论文超参数配置：编码器和解码器各 6 层；D=512；头数 h=8；每头 d_k=d_v=64；前馈网络中间维度扩充成2048维

注意：

1. 512 个数字通常没有"第一维就是主语、第二维就是人称"这样的固定解释；它们共同表示特征
2. 层与层、头与头有各自的参数W_Q,W_K,W_V；同一头内，不同 token 位置共用同一套参数W_Q,W_K,W_V（最后生成的QKV不同是因为本身的向量里面的数据不相等）
3. 嵌入表和线性层的权重从头训练时通常随机初始化

1.3 最后的 512 维向量怎样变成英文预测?

交叉熵损失。类似"花卉预测"任务中最后对花卉种类的识别，最后的512维向量通过全连接层对词汇表里的每个单词生成相应分数，在经过sigmoid函数变成每个词汇的概率（所有词汇的概率和为1）

1.4 损失函数比较什么?

比较的是"对词表中各 token 的预测"和"这个位置正确 token 的编号"（交叉熵损失），不是两个 512 维嵌入向量的距离。

解码器输入：<开始> I like cats
正确标签： I like cats

对每一个已知位置的下一个未知位置进行预测，对最后输出的词汇概率表拿出正确单词的概率p，进行loss'=-log(p)的操作，最后将每个预测位置的损失loss'相加就得到了总损失。

（正确 token 的预测概率越高，这个位置的损失越小）

1.5 怎样优化?

流程：前向传播得到预测 → 计算损失 → 反向传播得到梯度 → Adam 更新参数

使用**Adam优化器**：会记录梯度下降的方向（不会产生忽左忽右的震荡）；会随着历史梯度的大小动态调整学习率（梯度变小时把学习率调低）

注意：各个注意力子层也不需要各自单独设置一个损失，仅用对最后的损失进行梯度前向传播即可对参数进行更新

2. 所有注意力共用的数学原理

注意力：在融入A与B（内含b1,b2,b3）的相关性时，通过权重来表现A对b1,b2,b3的关注度大小（'A分别对这三个数据有多关注？'）

2.1 Q、K、V 是什么?

名称	通俗理解	用在哪里
Q，Query，查询	当前对象想找哪些位置（一种倾向，对不同位置有大小之分）	想融入与其他数据相关性的"主体"上，也就是"出发点"，最终数据一般就更新在他上面（强调的是对不同位置的倾向程度）
K，Key，键	这个位置给对象线索的匹配程度	保存在被查询对象中，从被查询对象的角度出发看里面的内容与查找对象是否匹配
V，Value，值	这个位置实际提供什么内容	与前两者的"关注"不同，这里记录的是实实在在的具体内容信息，之后乘上权重被关联到"出发点"中

Q、K、V 是向量，不是一个"查询分数"。Q 和 K 相乘之后，才得到一个匹配分数表。

2.2 W_Q、W_K、W_V 怎样生成 Q、K、V？& Q、K 怎样变成权重表 A？

W_Q . W_K . W_V → 一套对token键值查询的计算规则 .

形式<以W_Q为例>:

$$W_Q = \begin{bmatrix} a & k & & & \\ b & l & & & \\ c & m & & & \\ d & n & & & \\ \vdots & \vdots & & & \end{bmatrix}$$

↳ 一套计算规则(有n列就有n套计算规则)

对于∀token_i = $[x_1, x_2, x_3, x_4 \cdots]$: (假设只有两套规则)

$token_i_Q1 = ax_1 + bx_2 + cx_3 + dx_4 + \cdots$

$token_i_Q2 = kx_1 + lx_2 + mx_3 + dx_4 + \cdots$

所以token_i 的 $Q_i = [Q_①, Q_②]$

在同一子层中的各个token也用相同的计算规则(W_Q)

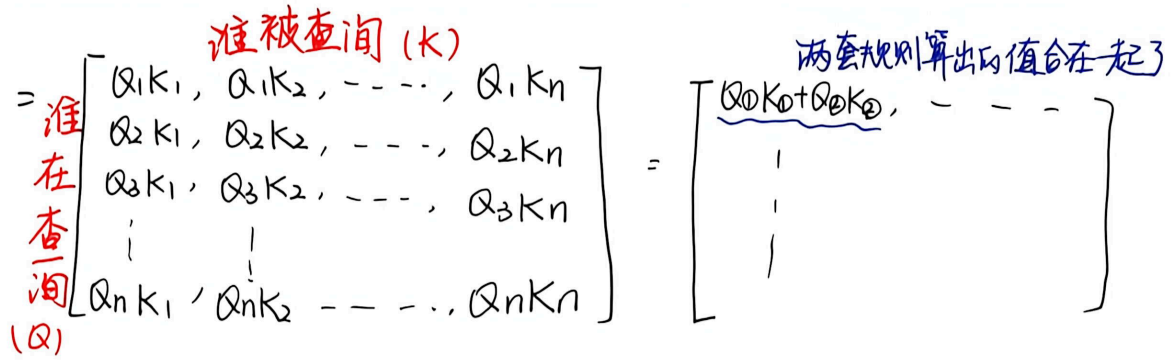
同理各个token根据规则生成各个 $[K_①, K_②]$. $[V_①, V_②]$

* Q.K.V的规则个数也保持相等

<也就是一套规则有自己的W_Q.W_K.W_V>

生成注意力权重表A: → $[Q_①\ Q_②]$

$$Q \cdot K^T = \begin{bmatrix} Q_token1 \\ Q_token2 \\ \vdots \\ Q_token\,n \end{bmatrix} \cdot \begin{bmatrix} K_token1, & K_token2, & \cdots, & K_token\,n \\ \downarrow & & & \\ \begin{bmatrix} K_① \\ K_② \end{bmatrix} & & & \end{bmatrix}$$



(QK^T)→代表各个数据间的关联程度

*为什么使用多套规则：让浅层的数据生成Q、K、V的表达能力更强

后续：QK^T → 除以$\sqrt{d_K}$（防止）→ 经过softmax函数 → A（注意力分数表）

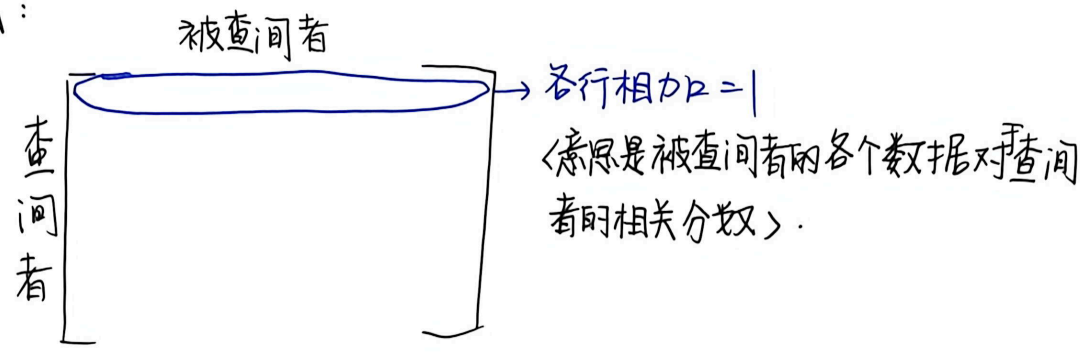
防止数据尺度过大（向量512个数据过多，而让Q_iK_j尺度很大），

导致在softmax函数中变化成分数的效果不好，

↳指数函数后归一化

故先用了一步确保各层尺度统一。

A：



〈意思是被查询者的各个数据对于查询者的相关分数〉。

（因为举例是自注意力，所以被查询者中也包括自己本身）

2.3 Z 代表什么?

计算：A里的各个元素与对应的被查询的z相乘后累加

例如一行权重是 [0.2,0.7,0.1]：　　z=0.2v1+0.7v2+0.1v3

Z 是融合了各位置信息的表示，不是已经预测好的下一个单词，也不是对未知 token 的嵌入向量。

（准确来说其实每一层就是融入了某某相关信息，代表着模型在这一步需要关注×××地方，后续反向传播时可以进行调整关注力度）

注意：

1. 最开始W_Q,W_K,W_V内的数值是随机生成的，而后通过梯度下降不断调整
2. q_i 和 k_j 是多维向量，分数取决于它们的方向及各分量如何配合；不是把某个 k“变大”，就能单独控制所有关系。
3. V 有 dv 个分量，该头的 zi 就有 dv 个分量；不会因为AV过后而新增添了位置（n*512形式不变）
4. **A 是当前输入重新计算的权重，不是模型提前存好的“词与词关系表”；换句子或换上下文就会重算。**

3. 多头：同一份输入，多套观察方式

（各个头唯一的区别就是最开始随机的W_Q,K,V数据不同）

多个头用不同的参数并行做注意力，再把各自汇总的信息组合起来。

最开始，将每个token相同的512维信息交给多个相同规则的头进行整合，每个头对512维向量压缩成64维，用自己初始随机化的W_Q,W_K,W_V进行计算Q,K,V，最后输出自己对与想关注信息（关注的内容一致）的融合表示。然后再将各个头的输出表示经过全连接层，将8*64维输入映射成512维输出（怎么读取输入的信息是通过模型的训练自己训练出怎么读取）

后续 训练时每个头对自己生成的W_K,W_Q,W_V进行梯度下降训练，形成了不同的观察方式后将根据自己训练出的观察方式输出融合。

在模型投入使用时相当于模型结合了各个头独立的观察方式进行信息间的融合。

以长度为 T 的自注意力输入为例：

```
输入 X：T × 512
    |头1：读取全部512维，用自己的W生成64维Q/K/V → T × 64     （这里又是通过一次全连接层把QKV三个512维向量投影成64维向量）
    |头2：读取全部512维，用自己的W生成64维Q/K/V → T × 64
    |……头8                                    → T × 64
        ↓ 沿特征维度拼接
        T × (8×64) = T × 512
        ↓ 乘 W_O（512 × 512）---------------全连接层（自己通过训练来判断哪些数据要结合）
        T × 512
```

不是把原始 512 个数直接瓜分成 8 份。每个头都能读取完整输入，再通过自己的参数得到 64 维表示。

多头的意义：让模型尽可能多的关注到数据间的不同关联（因为初始数据不同，每个头会算出各自的权重，有可能得到自己独特的关注方式，所以要想办法把这些关注点整合，让数据间的关系观察的更充分）

注意：

1. 8 个头不是 8 个独立训练的模型；**它们属于同一层**，通过模型最终损失共同训练。
2. 各头维数相同，不代表数值相等、含义相同或重要性相同。
3. 不同头可能学到不同关注方式，**但不能保证每个头都有清晰的“语法头”等名称。**
4. 从头训练时，各头的参数通常随机初始化。

4. 三种注意力：算法相同，只是QKV的位置以及是否掩码不同

类型	Q 来自哪里	K、V 来自哪里	允许看哪里	每头 A 的形状
编码器自注意力	编码器当前层输入 X	同一组 X	所有有效原文位置	n × n
解码器掩码自注意力	解码器当前层输入 X	同一组 X	当前及之前的解码器输入位置	m × m
解码器交叉注意力	掩码子层处理后的解码器输出 H	编码器最终输出 E	所有有效原文位置	m × n

4.1 编码器自注意力

原文中的各位置参考同一句话中各个位置的信息（包括他自己），形成带上下文的表示。

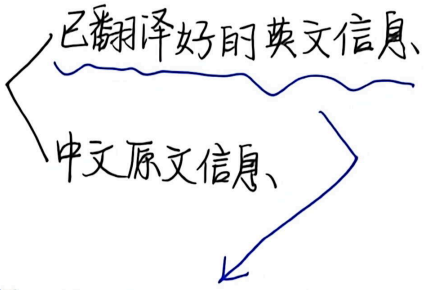
（先生成注意力表，再结合每个位置的value进行结合）

（之前注意力共同数学原理示例中列举的就是自注意力，看该处即可）

4.2 掩码自注意力

多了掩码一步：整理解码器已有的目标语言前缀，禁止它利用未来答案。

掩码自注意力

使用逻辑：生成新词语时要参考 

对这一步的原始相关信息就需要用到

① 在模型训练完投入使用时：

每次预测出一个英文单词预测下一个单词时.

(like)

已翻译好的：["开始", "I"] —添加"like"→ ["开始", "I", "like"]

作为已翻译好的英文信息.

② 而在模型还在训练时：

比如在预测"like"时，理论上是不能知道"like"及以后的信息的，故要进行掩码

原文本：["I", "like", "cats", EOS] → 论此时<也要预测什么时候结束>

目标

处理后的文本：["开始", "I", "like", "cats"]

每个位置实际上预测的是后面的位置是什么

(意思就是到"I"时已知"I"及以前的内容，来预测"I"之后的未知token)

而后续如何处理未知信息不"泄露"就是将A的相关分数设为 $-\infty$ 即可

三个位置的掩码：

	可被查看的输入位置		
	<开始>	I	like
<开始>发起查询	0	$-\infty$	$-\infty$
I 发起查询	0	0	$-\infty$

like 发起查询　　0　　0　　0

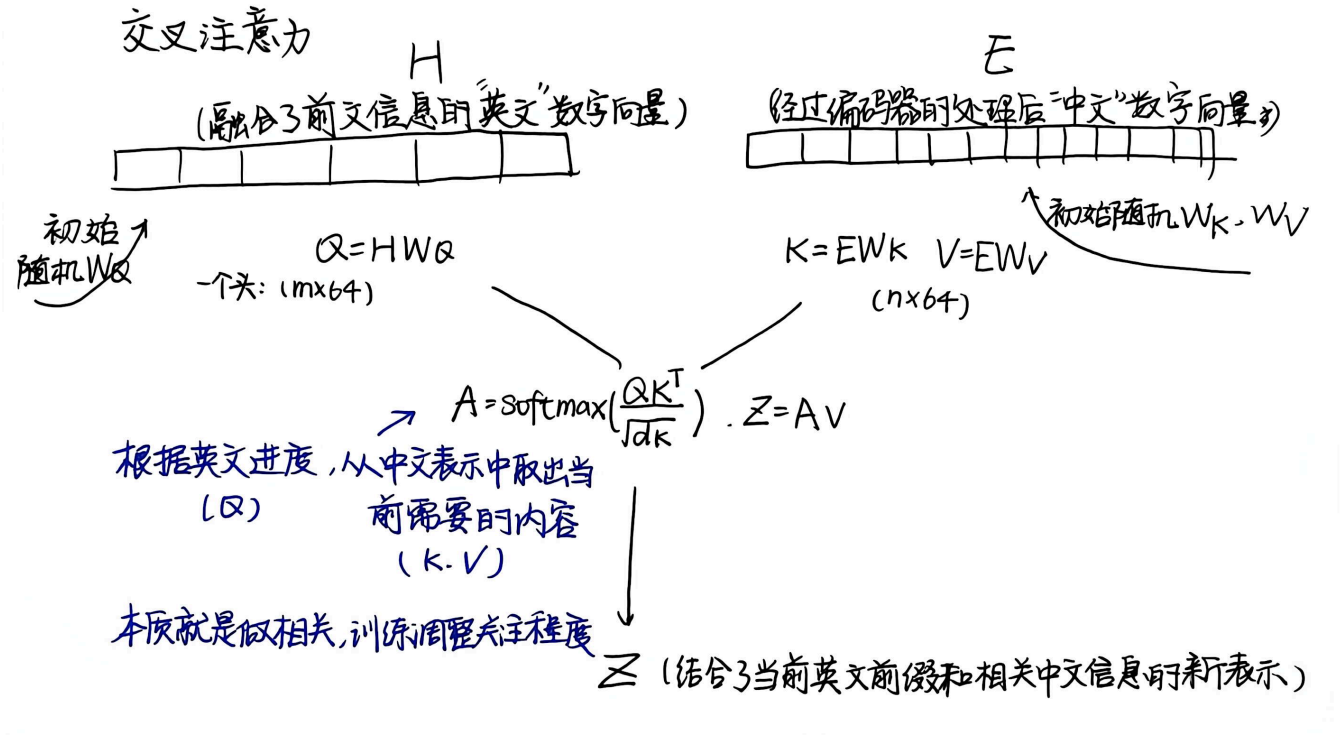
（0代表有实际注意力数字）

注意：

1. 输入中的 l 位置预测的是 like，允许它看 l 没有泄露答案；要预测 l 的位置只看到 <开始>。
2. 这一步不直接使用编码器输出；原文信息由后面的交叉注意力提供。
3. 掩码本身不训练；W_Q、W_K、W_V、W_O 都会通过最终损失训练。没有局部损失，不代表没有梯度。

4.3 交叉注意力

解码器拿着"英文已经生成到哪里"的状态，去查询编码器整理好的中文信息。



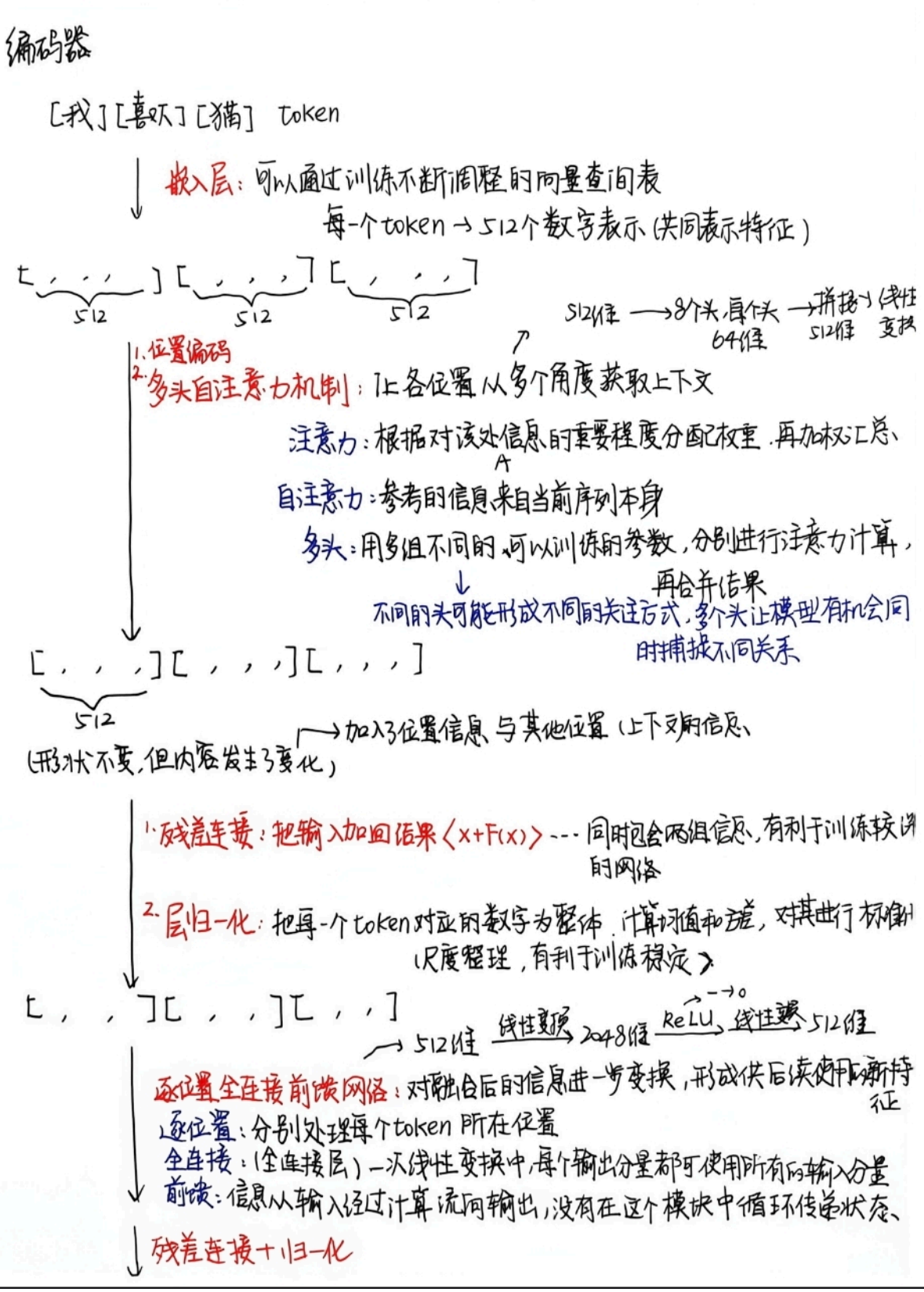
A表示从原文位置 j 读取信息的权重。结果是各目标位置从原文汇总来的信息，还不是最终词表预测。

注意：

1. 原文有 n 个 token、译文有 m 个位置，可以 n≠m；交叉注意力就是用 m×n 的表连接两边。
2. 它通常不需要遮挡原文的后面部分，因为整句原文已经可用；但需要忽略补齐用的 PAD。
3. 中文词和英文词不必一一对齐；模型需要结合词义、上下文和语序组织译文。

5. 编码器和解码器的内部架构：每步的数据结构

5.1 编码器



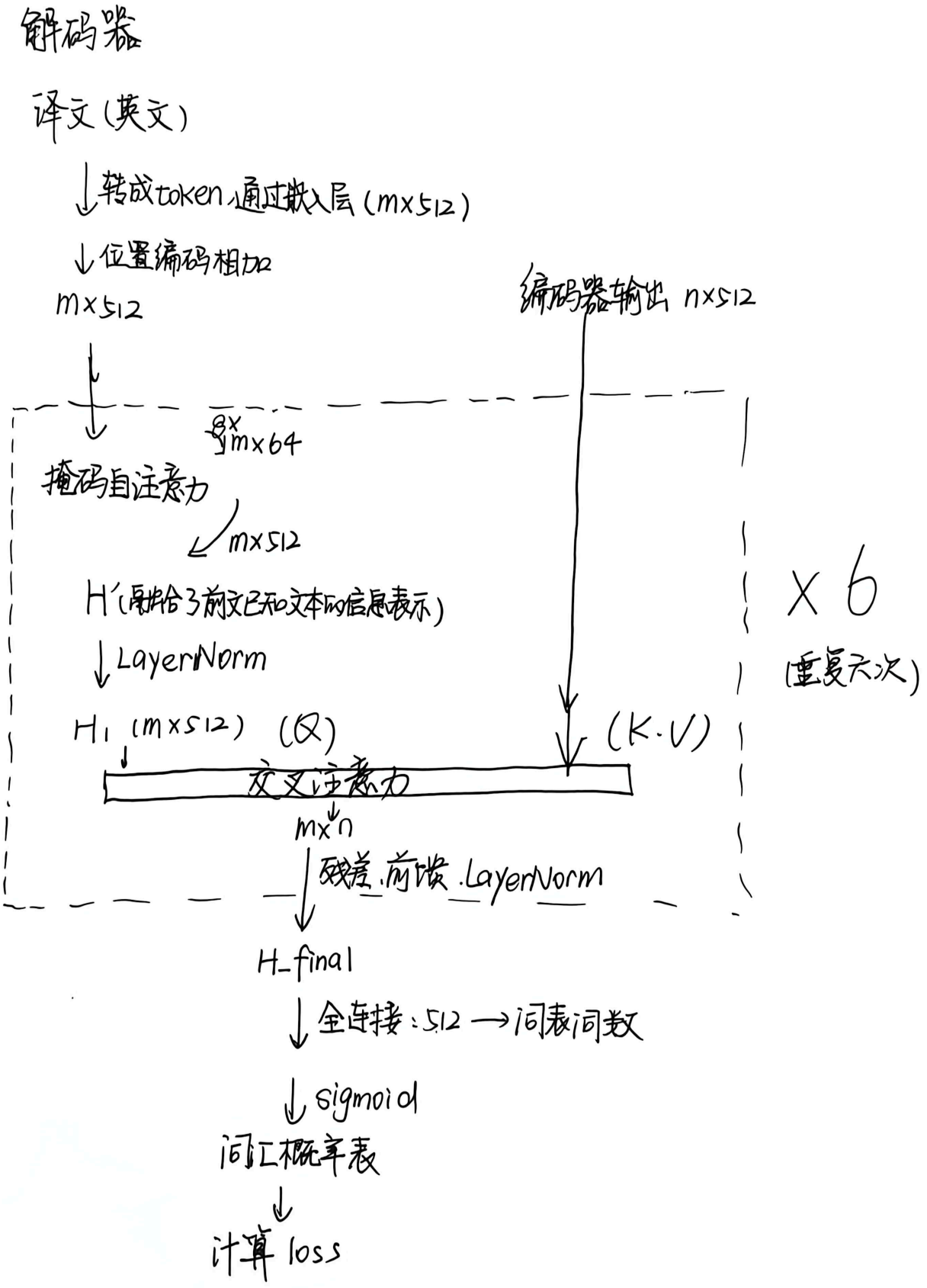
补充：在多头自注意力层中要重复六次，使数据能充分融合周围位置的信息（下图为真实结构）

```
┌── 编码器共重复6层 ──────────────────────────────
| 多头自注意力：让原文各位置互相汇总信息
| 每头 n×64 → 拼接 n×512 → 输出投影 n×512  ----------------------------每次在一个注意力子层就分开八个头最后合并
|     ↓ 加回该子层输入，再做 LayerNorm
| H1：n×512，融合注意力结果并保留输入路径
|     ↓ 逐位置前馈网络：加工每个位置内部的特征
| n×512 → n×2048 → ReLU → n×512
|     ↓ 加回前馈网络输入 H1，再做 LayerNorm
| 本层输出：n×512，作为下一层编码器的输入
└──────────────────────────────────────────────────

    ↓ 最后一层输出
编码器最终表示 n × 512
```

（形状不变，内容不断变化）

5.2 解码器


解码器
译文(英文)
转成token,通过嵌入层(mx512)
位置编码相加
mx512
编码器输出 nx512
8x
mx64
掩码自注意力
mx512
H'(融合了前文已知文本的信息表示)
LayerNorm
H1 (mx512)
(Q)
(K.V)
交叉注意力
mxn
残差.前馈.LayerNorm
X 6
(重复六次)
H_final
全连接:512 → 词表词数
sigmoid
词汇概率表
计算 loss

6. 三个配套部件：残差、归一化、前馈网络

6.1 残差连接：保留输入路径，再加上这一层算出的变化

公式： **y=x+F(x)**

意义：让数据不要忘了之前的信息；同时让backward时回传路径更直接简单

注意：这里的 x 是当前子层的输入，不一定是最初的 token 嵌入。x 与 F(x) 需要形状相同，所以子层最终通常回到 512 维。

6.2 层归一化：逐个 token 整理数值尺度

先整理这个 token 内部的数值尺度，再用可训练的缩放 γ 和偏移 β 调整，有利于稳定训练。

特别强调：最终输出 y 不一定均值为 0、方差为 1；γ和β都是可训练的参数 。

注意：不是把整句话所有数字混在一起归一化，也不是在不同样本间计算。γ、β 是参数，μ、σ^2 根据当前 token 的向量计算。论文采用"残差相加后归一化"的顺序。

6.3 逐位置前馈网络：对每个token的全连接层，独立加工每个位置的特征

```
每个 token 的512维
→ 第一层全连接：扩到2048维
→ ReLU：逐个分量把负值变成0，引入非线性
→ 第二层全连接：回到512维
```

W_1 的形状为 512×2048，W_2 为 2048×512。扩展后的特征经过非线性加工，再组合回模型宽度。

注意：

1. 每个位置独立处理，但同一层共用同一套 W_1、W_2；没有直接把不同 token 混在一起。
2. 注意力负责跨位置汇总信息，前馈网络负责加工该位置内部的特征。
3. 输出仍是该位置的表示，不保证某个数字对应一个可直接说出的语义标签。

7. 技术细节

7.1 位置编码的加入

在编码器和解码器堆叠层的最底部，把位置向量与 token 嵌入逐元素相加。

1. 两部分都是 512 维，相加后仍是 512 维。不是拼成 1024 维，也不是每经过一层就重新加一次。
2. 位置编码让模型能利用顺序信息，否则仅靠 token 内容不容易区分"猫追狗"和"狗追猫"。
3. 论文用不同频率的正弦、余弦计算位置编码（好像是波段信息？有点没看懂，不过因为是固定公式，现阶段先学会套用）

7.2 掩码注意力输出的右移

完整的训练安排：

输入位置上的 token	掩码允许利用的完整前缀	这个位置的正确标签
`<开始>`	`<开始>`	I
I	`<开始> I`	like
like	`<开始> I like`	cats
cats	`<开始> I like cats`	`<EOS>`

输入位置与要预测的目标位置错开一格。I 位置预测的是 like，并非“拿 I 来预测 I”。

（预测 cats 时，不能读取未来输入位置上的 cats；预测 `<EOS>` 时，cats 已经是合法前缀，所以必须让它进入输入）

7.4 译文字符数m在实际翻译与模型训练中的区别

训练时，m 是已知量，由目标序列构造决定；生成时，m并不重要，最后翻译到截止符后m的个数自然知道

训练中，右移输入和正确标签长度相同，所以每个输出位置都有对应标签；原文长度 n 可以与 m 不同。

7.5 模型训练结束-------< EOS >

词表通常包含 `<EOS>` 这样的特殊 token。训练时把它放在正确译文末尾，让模型学习何时预测它(所以最后的概率表中也包含< EOS >的概率)

句号是普通文本内容，不一定等于序列结束。沿用分开的简化 token 例子：

```
输入：<开始> I like cats .
标签：I like cats . <EOS>
```

生成 EOS 后停止；程序还会设置最大输出长度。模型参数并不提前指定这句话必须输出几个英文 token。

7.6 生成译文时训练能并行，生成却要一步一步：

训练时完整正确译文已知，可以一次构造所有错位输入；掩码阻止各位置偷看答案。生成时后一个输入取决于刚生成的 token，所以需要逐步进行。

7.7 在训练完成投入使用时注意力怎样处理没见过的词语组合：

它学习的是共享的变换参数，不是把所有句子的注意力权重存进表里。新输入到来时重新算 QKV 和 A。

新组合可以利用已学到的特征计算关系；不在词表里的完整词通常由分词器拆成已有片段，或按分词方案处理未知项。能计算不代表一定翻译正确，效果仍取决于训练和输入。

7.8 dropout 和论文中的归一化顺序

论文在子层输出上用 dropout，再残差相加、LayerNorm；嵌入与位置编码相加后也使用 dropout

dropout 用于训练，实际预测时关闭。

7.9 跟nanoGPT 时不要把两种结构差异：

原论文的翻译模型有编码器和解码器，所以有交叉注意力；GPT/nanoGPT 没有这个独立编码器，因此没有相应的交叉注意力。

（投入原始文本时没有原始文本需要处理，只有前文）